NORSK DATA ND-100 FAMILY, SINTRAN III

SINTRAN boot and segments: how the operating system gets from a disk pack into memory

A guide for people new to the machine. It follows the load button to the @ prompt, shows what sits where on a pack and in memory, and explains the one thing that trips everyone: the three copies of the resident data page and why a CPU number change has to reach two files.

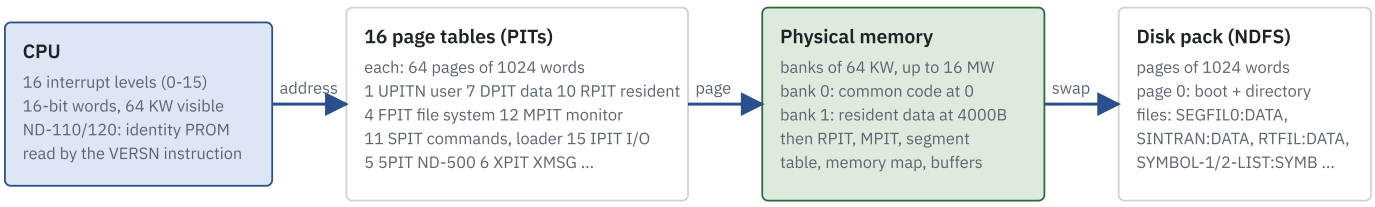
VERIFIED read off real packs or seen in a CPU trace on 2026-10-29 SOURCE from the SINTRAN NPL source, the generation listing or an ND manual OPEN not checked yet

1 The machine in one picture
2 Eight words you need
3 What is on a system pack
4 The boot, stage by stage
5 Memory after boot: the page tables
6 The three copies of the data page
7 Finding a segment from the pack alone
8 The system segments, by job
9 Where the CPU number sits
10 Looking at a pack yourself
11 Where to read on

1

The machine in one picture

An ND-100 (and its faster siblings ND-110 and ND-120) is a 16-bit minicomputer. A program sees 64 K words; physical memory is up to 16 M words in banks of 64 K words. Between the two sits the memory management system: **16 page tables** of 64 entries each, every entry mapping one 1024-word page. SINTRAN III gives each table a job and switches tables as it moves between the user, the monitor, the file system and the drivers. Disks are divided into 1024-word pages too, and every disk carries an NDFS directory of named files.



The one idea to hold on to

Nearly everything outside the small common code is a segment: a block of up to 64 KW that has a number, a place on the disk, a length, and a page table it is mapped in while it runs. The kernel's own code and data are segments; so is every program. One segment table describes them all. Boot is "load the right segments into the right page tables"; running is "swap segments in and out of those tables". Once that is clear, the rest of this page is detail.

Eight words you need

segment

A numbered block of up to 64 KW of code or data that SINTRAN swaps as a unit. It has a logical base inside the 64 KW space, a length in pages, a disk page in a segment file, and a page table it runs in. Names like S3FS are given by the RT-loader.

page table (PIT)

One of the 16 tables of the memory management system. SINTRAN uses each for a role: 7 DPIT for the resident data, 10 RPIT for resident code, 12 MPIT for the monitor and drivers, 4 FPIT for the file system, 11 SPIT for the command processor and loaders, 1 and 2 for the user.

segment file

One contiguous NDFS file holding segments page after page. File 0 is `SEGFIL0:DATA`. "File 4" is the kernel's name for an absolute save area at disk page 1, which is inside `SINTRAN:DATA` and `MACM-AREA:DATA`. Files 1-3 exist only if the installer made them.

segment table

8 words per segment: links, LOGAD (base page), SEGLE (length), MADR (page in its file), FLAG (file number in bits 13-15), status, core-map head. Lives in memory, and on disk as segments 27 (image) and 24 (save) inside `SEGFIL0`.

DPIT, the resident data page

The data every part of the kernel shares: RT descriptions, device datafields, system variables, the CPU number (SYSNO), the segment-file tables. Logical 4000B in page table 7. Stored as segments 23 (memory), 44 (image on disk), 53 (save on disk).

save / image / running

Most system segments exist three times. The save copy is what the installer wrote; a cold start copies it over the image copy and loads it. The image copy is the working copy; a warm start loads it as it is. The running segment is the one in memory.

symbol list

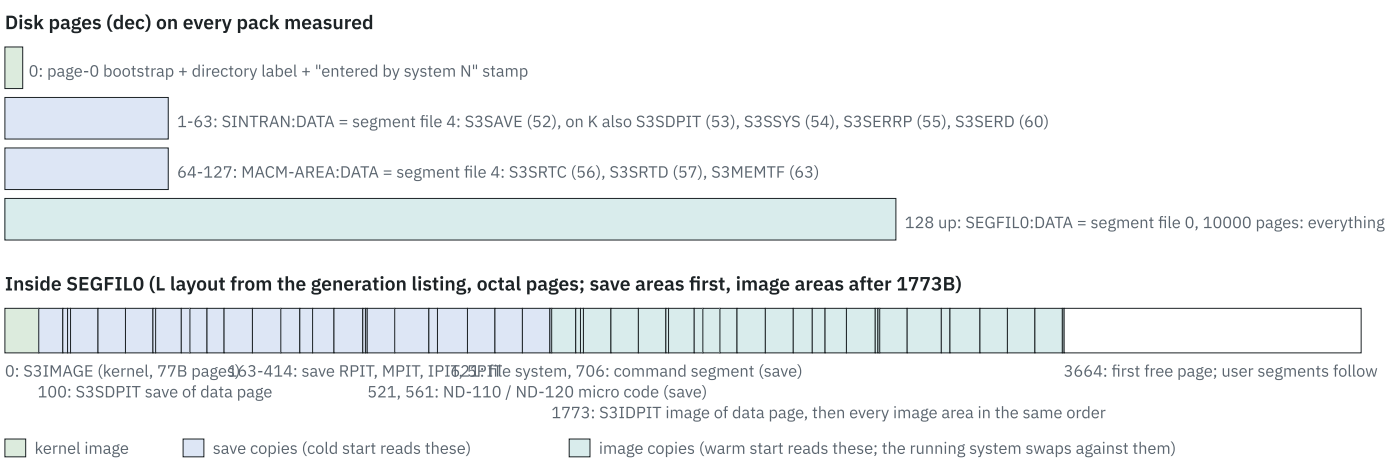
`(SYSTEM)SYMBOL-1-LIST:SYMB` and `SYMBOL-2-LIST:SYMB`: every kernel symbol of that build with its octal address (`SYSNO=004051`). Every system pack carries its own. Addresses move between builds, so tools read these, never a table.

RTFIL and the name records

The RT-loader keeps every segment name as a "DEFINED SEGMENT NAME" record (type 25B, the name packed as SIXBIT, the segment number) in `RTFIL:DATA` and in segment 10 `S3RTFIL`. That is where `S3SDPIT = 53` comes from.

What is on a system pack

Measured on real K and L packs `VERIFIED`: the kernel lives in three NDFS files at fixed disk pages, and inside `SEGFIL0:DATA` the generation job laid the system segments out page by page. The bar is the start of `SEGFIL0` on an L pack, drawn to scale (one unit = 8 pages, octal page numbers from the listing).



The order and sizes come from section 29.6 of the generation listing (`MSSLE` / `MISLE` macros), and the measured segment table of the L packs has the save data page at 100B = page 64 and the image at 1773B = page 1019, exactly as listed. On K the save copies of the data page, system segment and error program are in `SINTRAN:DATA` instead.

The boot, stage by stage

1

Firmware loads one page VERIFIED

Pressing LOAD (RetroCore: `boot bd0`; nd100x: `--boot=smd0`) runs the ND-100's mass-storage load: the firmware reads **disk page 0**, 1024 words, into memory at address 0 and starts the CPU at address 0. That is everything the hardware knows how to do.

The CPU trace of the K pack starts at address 0 with `PIOF`, the first word of that program. ND-06.014.2A §4.2.5.2.

2

The page-0 program loads the kernel SOURCE

Those 1024 words are the loader `RELOA` plus a disk driver. It reads **segment 2 `S3IMAGE`**, page 0 of `SEGFIL0:DATA`, 77B pages of "common / start / restart code", into memory, then jumps to `SINTR`. SINTRAN itself rewrites page 0 with the current loader at every start, so the bootstrap on a pack is always its own kernel's.

`PH-P2-OPPSTART.NPL` 3712-3766 (RELOA), 850-876 (the cells it fills and the rewrite). Its untouched copy is segment 52 `S3SAVE` at disk page 1; on the K pack the two are byte-identical.

3

SINTR sets the system up, paging off SOURCE

Entry at `SINTR`. It turns interrupts off, probes the CPU (SYSEVAL: NORD-10 / ND-100 / ND-110 / ND-120, floating point, instruction set), on an ND-110/120 reads the identity PROM (GCPUNR) and if the signature 52652B is there overwrites SYSNO, FCPUN and the CPU type in memory, scans memory, **reads the resident data page from disk into bank 1 at 4000B**, builds the page tables, finds the devices, sets up buffers and interrupt vectors. On a **cold start** it first copies every save segment over its image segment.

Data page from the save copy S3SDPIT when `HENTFLAG=0`, from the image copy S3IDPIT otherwise (lines 262, 283-284); the 32 save-to-image copies at 771-841; GCPUNR only when SYSEVAL found an ND-110 or later (313).

> **SINTRAN K stops here on a programmed PROM.** VERIFIED The K start-up code reads PROM bytes 6-7 and jumps to `IOF; WAIT` at address 2 when they read 52652B. Nothing is printed. RetroCore and nd100x present no signature until an identity key is set, so K and L packs both boot.

4

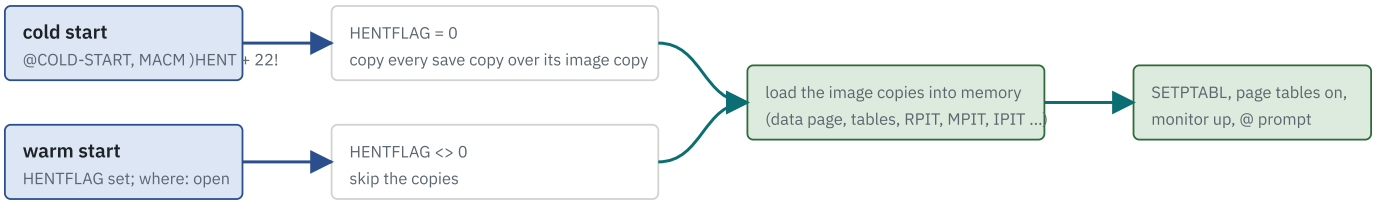
Paging on, interrupts on, the monitor runs SOURCE

`SETPTABL` loads or checks the ND-110/ND-120 control store from the micro code segment, maps the resident segments into their page tables and sets the page-table register on every level. Interrupts come on, the monitor starts on level 3 and the idle program on level 1. From now on the data page in memory is the live segment 23 `S3DPIT`.

5

Initial commands, banner, mode files VERIFIED

The start program runs the commands stored in the kernel buffer `INIBU`: `ENTER-DIRECTORY` of the system disk first, where the file system compares the directory's page-0 stamp with SYSNO and refuses with **3203 DIRECTORY ENTERED BY ANOTHER SYSTEM** if they differ; then `SET-AVAILABLE` and the mode files. The banner (`SINTRAN III - VSX/500 K`, CPU TYPE, CPU NUMBER) is printed from the data page. XMSG, the network servers and COSMOS start, and the `@` prompt appears.



MACM's)HENT is "GET SINTRAN FROM SAVE-AREA": despite the name it is the cold path. There is no third "HENT snapshot" area on disk.

Memory after boot: the page tables and what runs in them

Every table carries the small common code (pages 0-12B) so the kernel's shared routines are reachable whatever table is active, and every system code path uses DPIT (table 7) as its alternative table for data. From the K, M and N release information `SOURCE`.

PIT 1 UPITN user, normal

- the background or RT program's own segments
- on disk, "UPITN" in the generation marks copies that are not mapped at run time

PIT 2 UPITA user, alternate

- the program's second table (data, or a 128 KW program)
- XMSG kernel and XROUT run here (L and later)

PIT 3 FUPIT remote file user

- common code
- page 13: remote file access segment 22

PIT 4 FPIT file system

- common code
- page 13: the file system, segment 6 S3FS

PIT 5 5PIT ND-500

- common code
- 13: MON 60, segment 51
- 20: ND-500 system monitor, segment 30

PIT 6 XPIT XMSG

- common code
- 13: XMSG (K layout)

PIT 7 DPIT the data table

- 2: resident common data, 37 K: RT descriptions, datafields, SYSNO, segment-file tables (segment 23)
- 57: windows to background, ND-500, levels 10/12, 1/4
- 62: system segment window, 72: data segment
- 101: device name table (L and later)

PIT 10 RPIT resident code

- common code, 13: extended common (64)
- 15: monitor calls, B-level (level 4): segment 47
- generation 500 and later: segment administration

PIT 11 SPIT SINTRAN programs

- 13: edit routines (42), always
- 14: one of: command processor (3), RT-loader (4), DMAC (7), error program (14), service/mail (71), TADADM (36), servers (105-127)

PIT 12 MPIT monitor

- common code, 13: extended common
- 15: monitor level, level 14, drivers for levels 10-13, MPERF: segment 35

PIT 13 / 14 X5DP1, X5DP2 ND-500

- ND-500 name tables (21) and standard domains (20)
- last page: ND-500 monitor stack window

PIT 15 IPIT I/O (L and later)

- common code, 13: extended common
- 15: level 3 and the mass-storage level 11: segment 67

PIT 17 DTPIT direct tasks

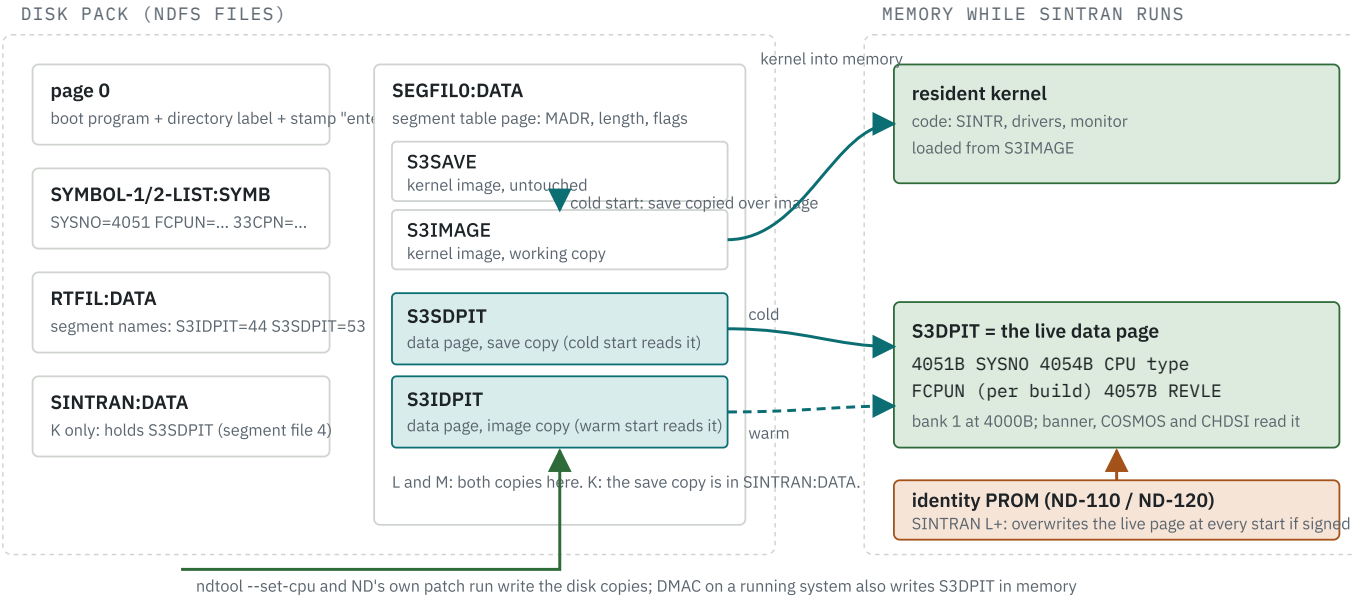
- direct tasks
- maps DPIT during start-up

Not in any table

- segment table, memory map
- RT register blocks and bitmaps
- big terminal datafields, ND-500 mailboxes, logical device tables

6

The three copies of the data page, and who writes which

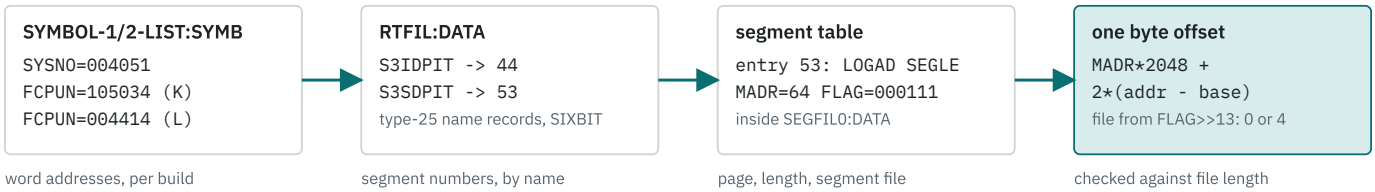


Why both disk copies: a cold start copies S3SDPIT over S3IDPIT and loads it, a warm start loads S3IDPIT as it is. A value in only one of them survives only one kind of start. ND's own install run patches S3SDPIT and then demands a cold start; the tool writes both so either start sees it.

7

Finding any segment from the pack alone, with no fixed numbers

Three things on every system pack are enough to find any kernel word on disk, whatever the SINTRAN generation: the symbol lists give the address, the RT-loader name records give the segment number, the segment table gives the page and the file. Nothing is hard-coded, because the numbers move between builds (FCPUN is 105034B on the K pack and 004414B on the L packs).



word addresses, per build | segment numbers, by name | page, length, segment file | checked against file length

The segment-table entry, 8 words

```
0 SEGLI link 1 PRESE link 2 LOGAD base page (low 6 bits) 3 SEGLE length in pages
4 MADR page in its file 5 FLAG (bits 13-15 segment file, bit 0 = 5OK) 6 SGSTA ring 7 BPAGL core map
```

The table page itself is found by shape: entry 0 all zero, 40 or more plausible entries, one entry whose MADR is the page it sits on.
Proven 2026-10-29: copies of the K and L packs set to CPU number 21238 this way boot with CPU NUMBER 21238 on the banner.

The system segments, by job

Numbers are octal. The names are the ones the RT-loader records give on the K and L packs; the jobs are the manuals' descriptions in plain words. Save and image copies are named with an S or an I after S3 (S3SDPIT, S3IDPIT). The full table with J to N differences is in chapter 30.

The kernel itself

2	S3IMAGE	the resident kernel image the bootstrap loads; 52 S3SAVE is its untouched copy
23	S3DPIT	the live data page; 44 image, 53 save
47	S3RPIT	resident code: most monitor calls; 25 image, 16 save
35	S3MPIT	monitor level, level 14, drivers 10-13; 26, 17
67	S3IPIT	level 3 and level 11 code (L and later); 66, 65
64	S3ECOM	extended common data (L and later); 61, 60
27	S3ISGT	the segment table on disk; 24 save
45	S3ISYS	system segment template; 54 save
111, 113	S3IU110, S3IU120	ND-110 and ND-120 micro code; 110, 112 save

Programs SINTRAN runs in SPIT and FPIT

3	S3CP (K: S3COM)	the command processor; holds INIBU; 13 save
6	S3FS	the file system; 12 save
4, 37	S3RTL, S3RTD	RT-loader code and data; 56, 57 save
10	S3RTFIL	RT-loader file image with the name records
7	S3DMAC	DMAC, the patching assembler
14, 5, 11	S3ERRP, S3ERRS, S3ERRL	error program, its system segment, error log; 55 save
42	S3ED	command-line edit routines, always on SPIT page 13; 41 image
71	S3SM	SINTRAN-SERVICE-PROGRAM and MAIL; 70 save
36	S3TAD	TAD administration (COSMOS remote terminals)
22	S3RFAC	remote file access
101	S3DNAM	device name table (L and later); 100 save

ND-500 / ND-5000, XMSG, servers

30	S3SM5	ND-500 system monitor; 62 save
51	S35PIT	MON 60 code; 50, 46
20, 21	S3SDT5, S3NMS5	ND-500 standard domains, name tables
76, 77	S3XMK, S3XROU	XMSG kernel and router; 74, 75 save (K: 33, 34)
103	S3XMFI	XMSG watchdog; 102 save
105, 107	S3INKSE, S3INKNA	NUCLEUS server and name server
115, 117	S3IERWC, S3IERWD	ERS watchdog code and data
121-127	S3IPRMA, S3IEVMS/S3IPWRS, S3IBOPC, S3IMTSE	processor manager, event message / PFTCON, BOPCOM, MT servers
43	S3PATCH	the addresses the patch mode file deposits into
63	S3MEMTF	memory-to-floppy dump

Above 127 the numbers belong to the generation and the RT-loader: two segments per background program, one per spooling queue, two per ND-500 process, and products loaded later (on the packs: COSPOOL, the Ethernet server ENCOSE0 with its buffers, XFTRAD, SNA3270, and some whose purpose is still open).

9

Where the CPU number sits, in one table

PLACE	WHAT	READ WHEN	WRITTEN BY
`S3DPIT (memory)`	the live data page: SYSNO, FCPUN, HWINFO(2)	all the time: banner, COSMOS local/remote test, CHDSI directory check	loaded at start; GCPUNR from the PROM on ND-110/120 (L and later); DMAC `)CLOAD S3DPIT`
`S3IDPIT (disk)`	image copy of the data page	warm start	cold start (copied from save); DMAC; ndtool
`S3SDPIT (disk)`	save copy of the data page	cold start	ND's install/patch run (`SYSNO`/`33CPN`); DMAC; ndtool
`SYMBOL-2-LIST 33CPN/33CPU`	the answers NEW-SYSTEM was given	the ND patch run, which deposits them into S3SDPIT	NEW-SYSTEM CPU-UPDATE; ndtool
`page 0, ext-info word 5`	system number that last entered the directory	ENTER-DIRECTORY: refused with 3203 if it differs from SYSNO	SINTRAN at enter/release; `ndtool --release` / `--set-cpu`
`identity PROM`	CPU number, type, legal users, signature 52652B	every start on an ND-110/120, SINTRAN L and later; K stops on it	the factory; emulator config (`cpu_number` and friends)

10

Looking at a pack yourself

Always work on a copy of a disk image. `ndtool` (repository norskdata-ndfs, folder `ndfs-c`) reads and changes all of the above from the pack's own files:

```
ndtool -i -v disk.IMG              directory label, extended info (system number, entered flag)
ndtool -t disk.IMG                 the files: SEGFIL0:DATA, SINTRAN:DATA, RTFIL:DATA, SYMBOL-1/2-LIST ...
ndtool --segments disk.IMG         every segment: number, name, page table, range, length, disk page, file
ndtool --export-segment 53 disk.IMG out.bin      copy one segment out (for carving or comparing)
ndtool --cpu-identity disk.IMG     SYSNO / FCPUN / CPU type / patch level in both copies, 33CPN, the stamp
ndtool -n --set-cpu 21238:9883 disk.IMG          preview a CPU number and type change
ndtool --set-cpu 21238:9883 disk.IMG             write both copies, 33CPN/33CPU, clear the stamp, read back
ndtool --release disk.IMG          clear only the "entered by system N" stamp
```

In RetroCore, `cpu config --cpu=ND120CX` boots a K or L pack unchanged; `Config cpu_number N` makes the emulated PROM override what the pack says, as a real programmed ND-120 would under SINTRAN L. DMAC on a running system does the same patch as `--set-cpu`, segment by segment.

Where to read on (NDInsight, folder SINTRAN/OS)

01-BOOT-SEQUENCE.md	the five stages with every source line, then the SINTR code step by step
11-RT-SEGMENTS-AND-SEGFIL.md	what a segment and a segment file are, the table entry, save / image / running
29-SEGMENT-MAP-FROM-THE-GENERATION.md	the generation listing decoded: which NPL file is in which segment, on which disk page
30-SYSTEM-SEGMENTS-AND-PAGE-TABLES-BY-RELEASE.md	every system segment and page table, J to N, with the manuals' descriptions
28-CPU-NUMBER-AND-TYPE-ON-DISK.md	the CPU number and type: measurements, the DMAC patch, the directory stamp, the tool spec
22-READING-RT-AND-SEGMENT-TABLES-FROM-MEMORY.md	the tables in memory and the name records (§6.5)
26-INITIAL-COMMAND-BUFFER-ON-DISK.md	the initial commands and the segment-table locator